

CS 342: Computer Vision and Pattern Recognition

Instructor: Br. Tamal Mj

Shanti Mantra

Lyrics in Sanskrit

Meaning in English

ॐ सह नाववतु।

Om, May we all be protected

सह नौ भुनक्तु।

May we all be nourished

सह वीर्यं करवावहै।

May we work together with great energy

तेजस्यि नावधीतमस्तु

May our intellect be sharpened (may our study be effective)

मा विद्विषावहै।

Let there be no Animosity amongst us

ॐ शान्तिः शान्तिः शान्तिः ॥

Om, peace (in me), peace (in nature), peace (in divine forces)

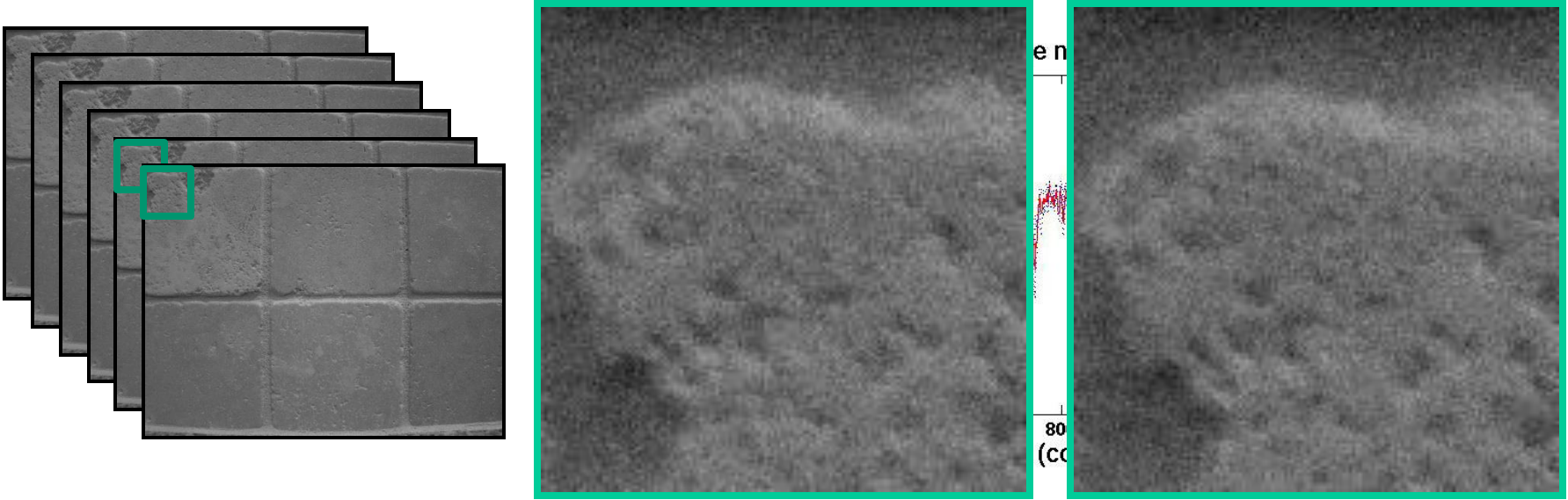
Plan for today

- Image noise
- Linear filters
 - Examples: smoothing filters
- Convolution / correlation

Main idea: image filtering

- Compute a function of the local neighborhood at each pixel in the image
 - Function specified by a “filter” or mask saying how to combine values from neighbors.
- Uses of filtering:
 - Enhance an image (denoise, resize, etc)
 - Extract information (texture, edges, etc)
 - Detect patterns (template matching)

Motivation: noise reduction



- Even multiple images of the **same static scene** will not be identical.

Common types of noise

- **Salt and pepper noise:** random occurrences of black and white pixels
- **Impulse noise:** random occurrences of white pixels
- **Gaussian noise:** variations in intensity drawn from a Gaussian normal distribution



Original



Salt and pepper noise

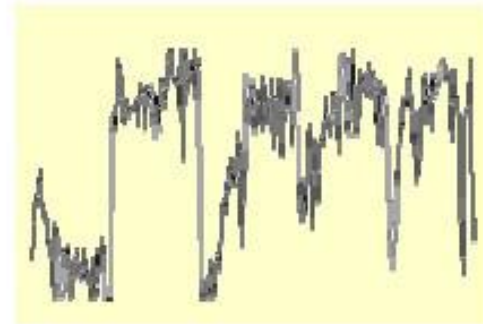
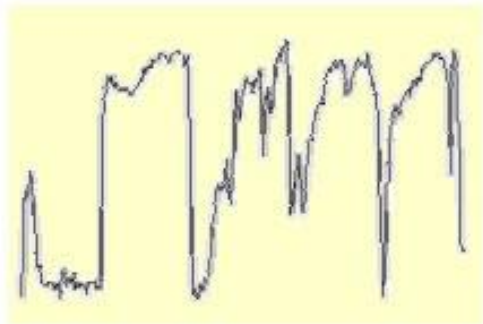
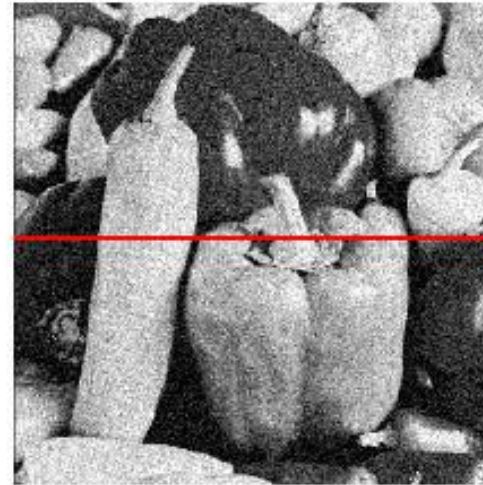


Impulse noise



Gaussian noise

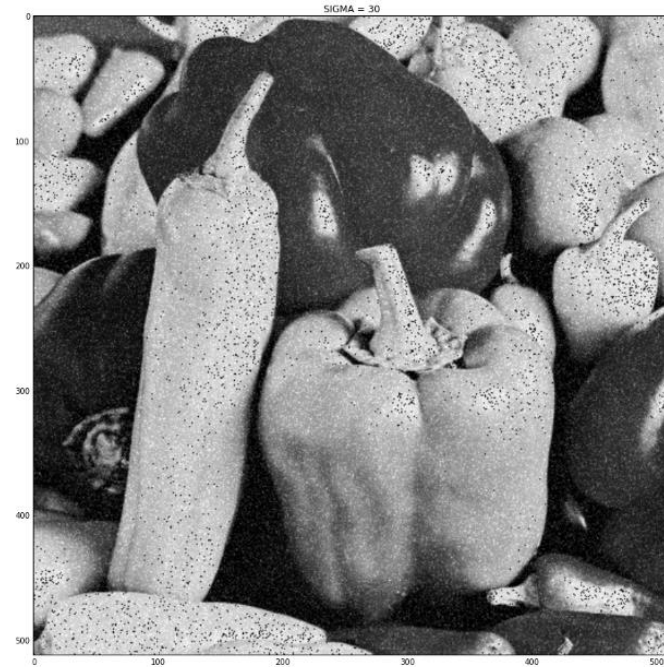
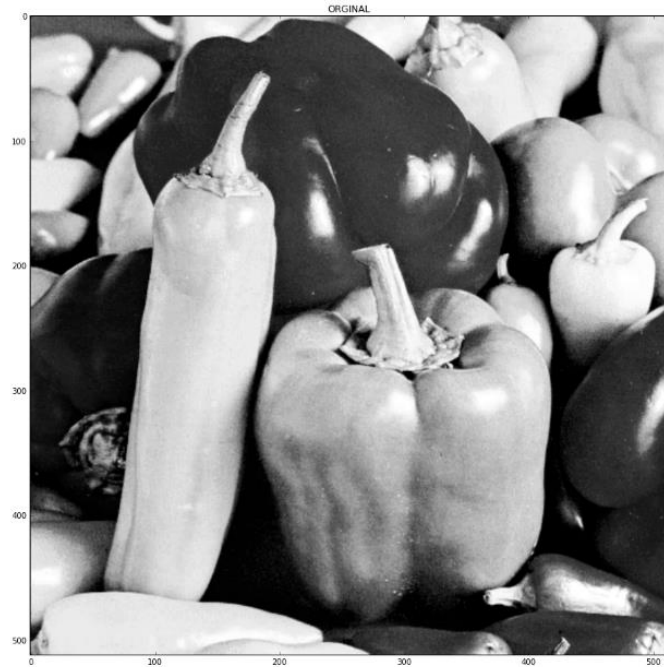
Gaussian noise



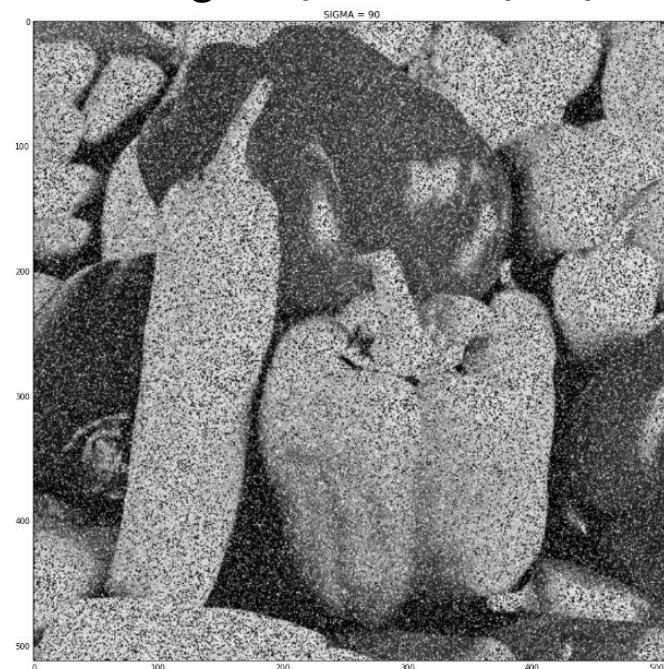
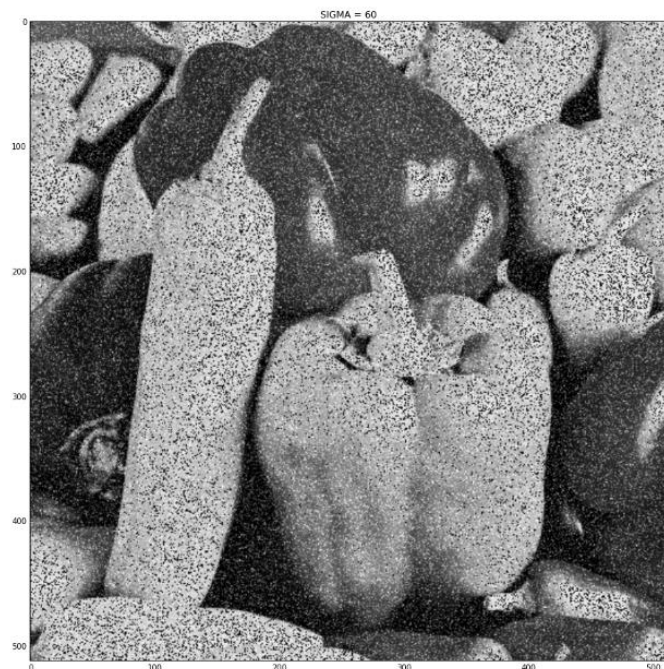
$$f(x, y) = \underbrace{\hat{f}(x, y)}_{\text{Ideal Image}} + \underbrace{\eta(x, y)}_{\text{Noise process}}$$

Gaussian i.i.d. ("white") noise:
 $\eta(x, y) \sim \mathcal{N}(\mu, \sigma)$

```
>> noise = np.zeros((height,width), np.uint8)
>> cv2.randn(noise, mean, ssd)
>> output = cv2.add(im, noise)
```



Effect of Standard Deviation: L2R, T2B: Original, SD = 30, 60, 90

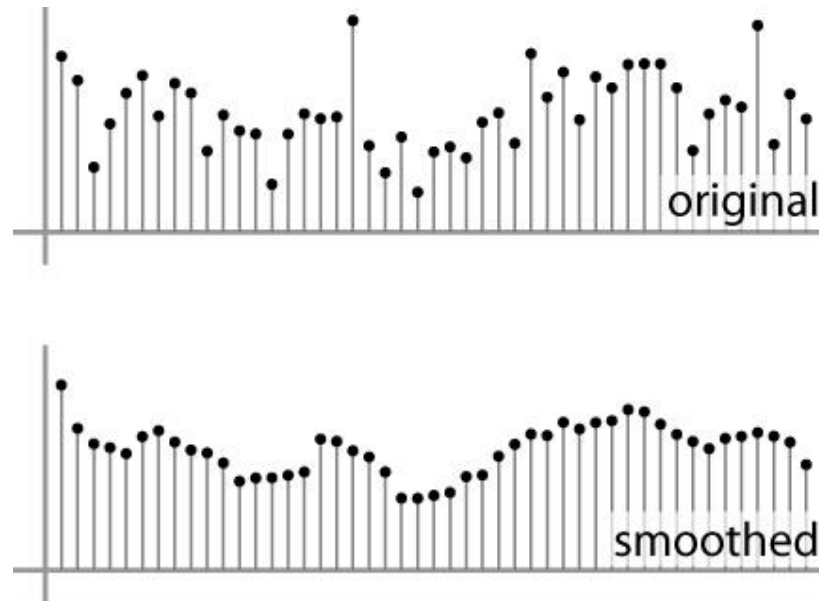


First attempt at a solution

- Let's replace each pixel with an average of all the values in its neighborhood
- Assumptions:
 - Expect pixels to be like their neighbors
 - Expect noise processes to be independent from pixel to pixel

First attempt at a solution

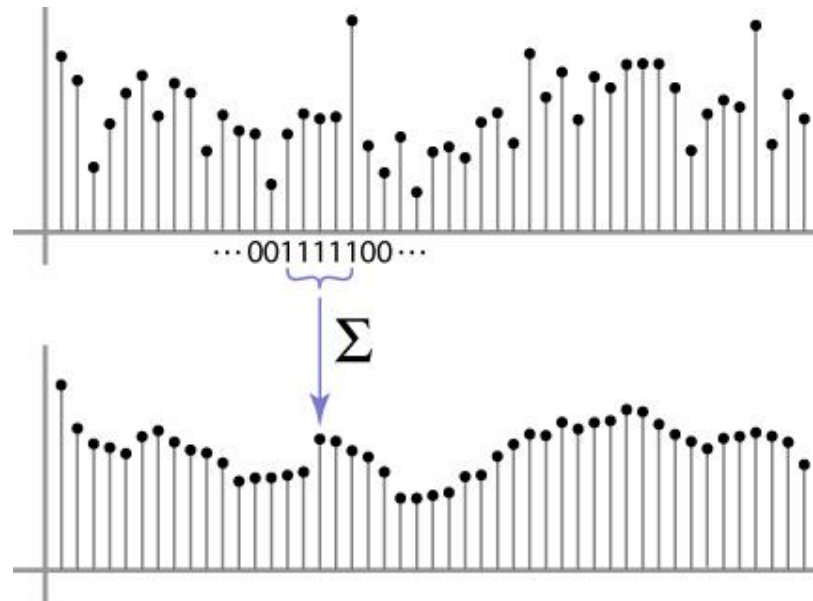
- Let's replace each pixel with an average of all the values in its neighborhood
- Moving average in 1D:



Weighted Moving Average

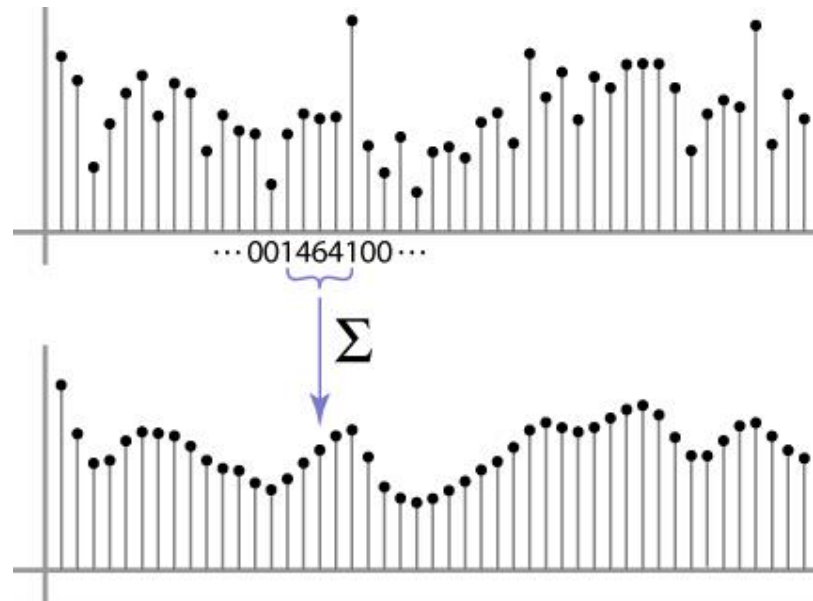
Can add weights to our moving average

Weights $[1, 1, 1, 1, 1] / 5$



Weighted Moving Average

Non-uniform weights $[1, 4, 6, 4, 1] / 16$



Moving Average In 2D

$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$G[x, y]$

	0								

Moving Average In 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10							

Moving Average In 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20						

Moving Average In 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20	30					

Moving Average In 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20	30	30				

Moving Average In 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

Correlation filtering

Say the averaging window size is $2k+1 \times 2k+1$:

$$G[i, j] = \underbrace{\frac{1}{(2k+1)^2}}_{\text{Attribute uniform weight to each pixel}} \underbrace{\sum_{u=-k}^k \sum_{v=-k}^k F[i+u, j+v]}_{\text{Loop over all pixels in neighborhood around image pixel } F[i,j]}$$

Now generalize to allow **different weights** depending on neighboring pixel's relative position:

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k \underbrace{H[u, v]}_{\text{Non-uniform weights}} F[i+u, j+v]$$

Correlation filtering

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

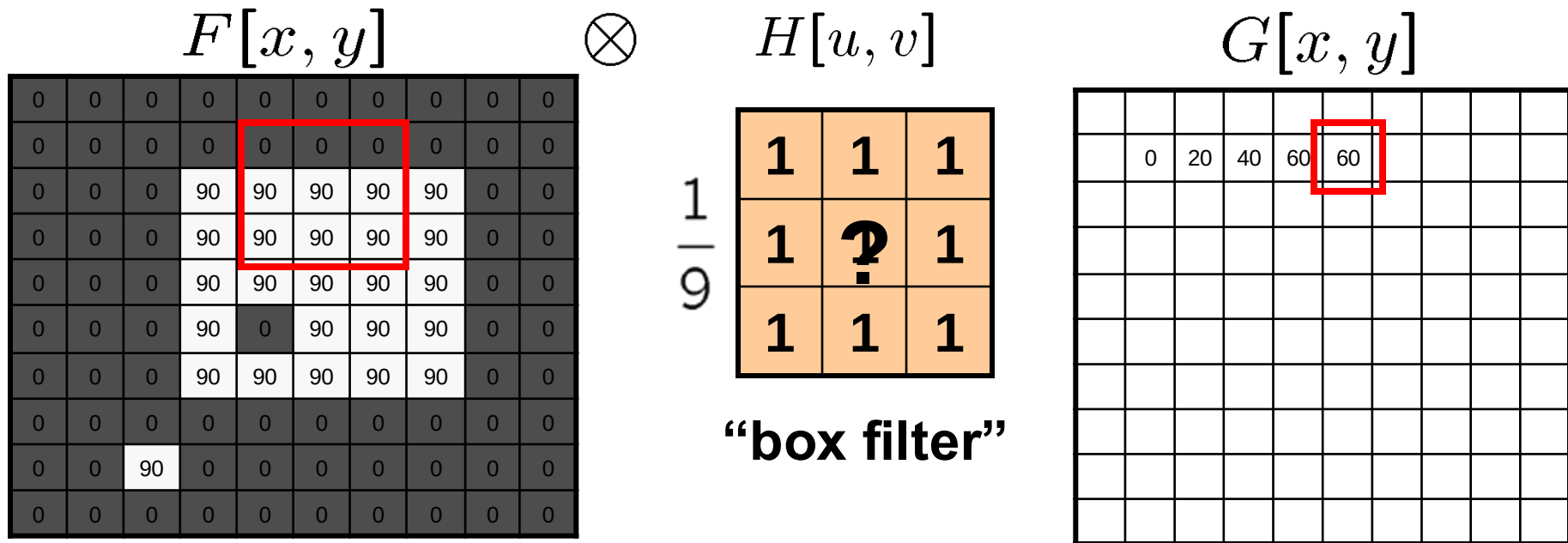
This is called **cross-correlation**, denoted $G = H \otimes F$

Filtering an image: replace each pixel with a linear combination of its neighbors.

The filter “**kernel**” or “**mask**” $H[u, v]$ is the prescription for the weights in the linear combination.

Averaging filter

- What values belong in the kernel H for the moving average example?



$$G = H \otimes F$$

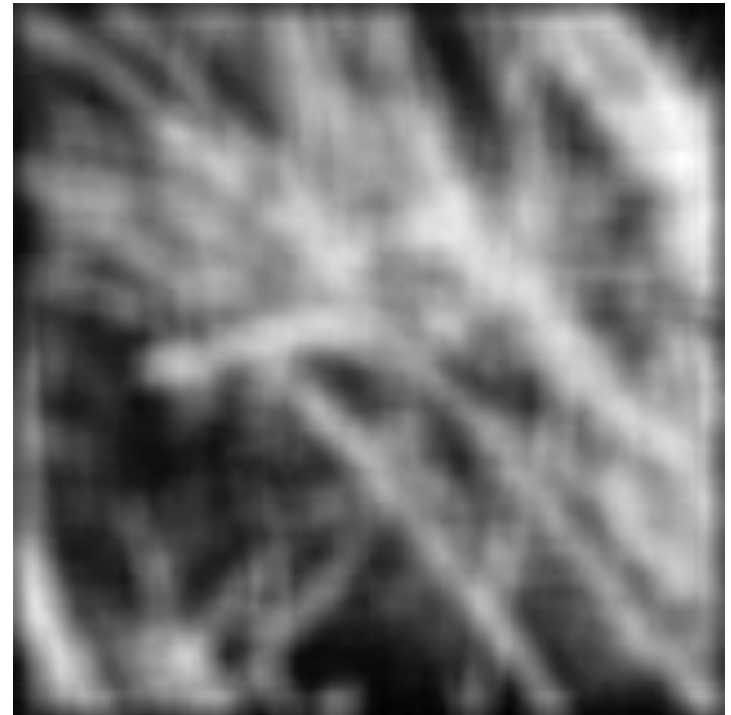
Smoothing by averaging



depicts box filter:
white = high value, black = low value



original

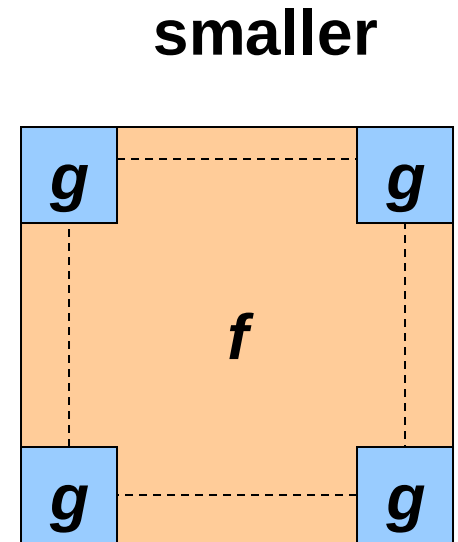
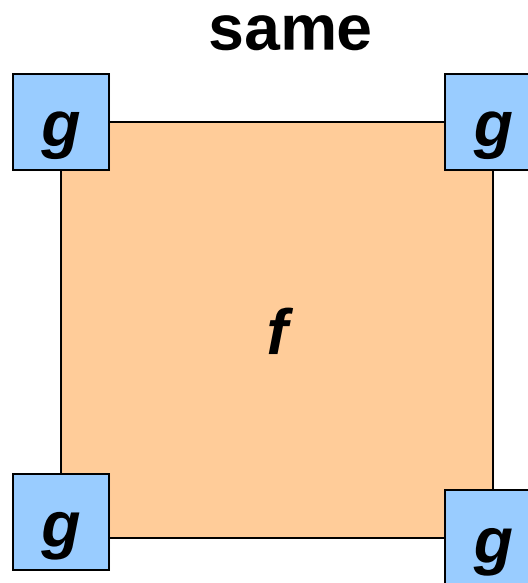
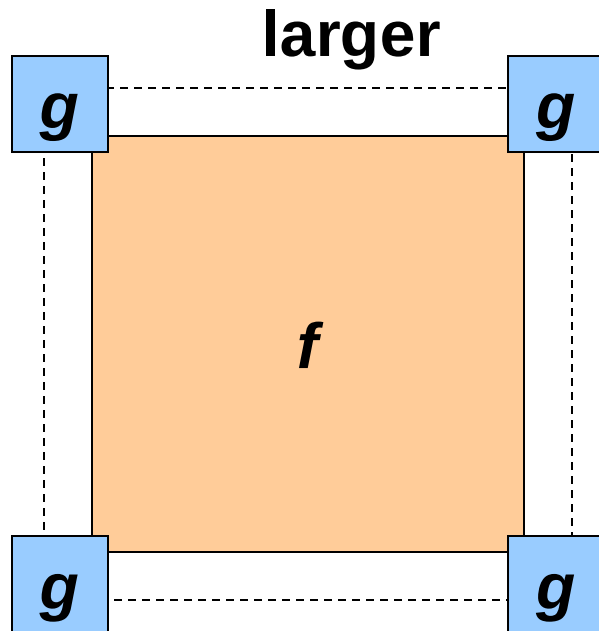


filtered

What if the filter size was 5 x 5 instead of 3 x 3?

Boundary issues

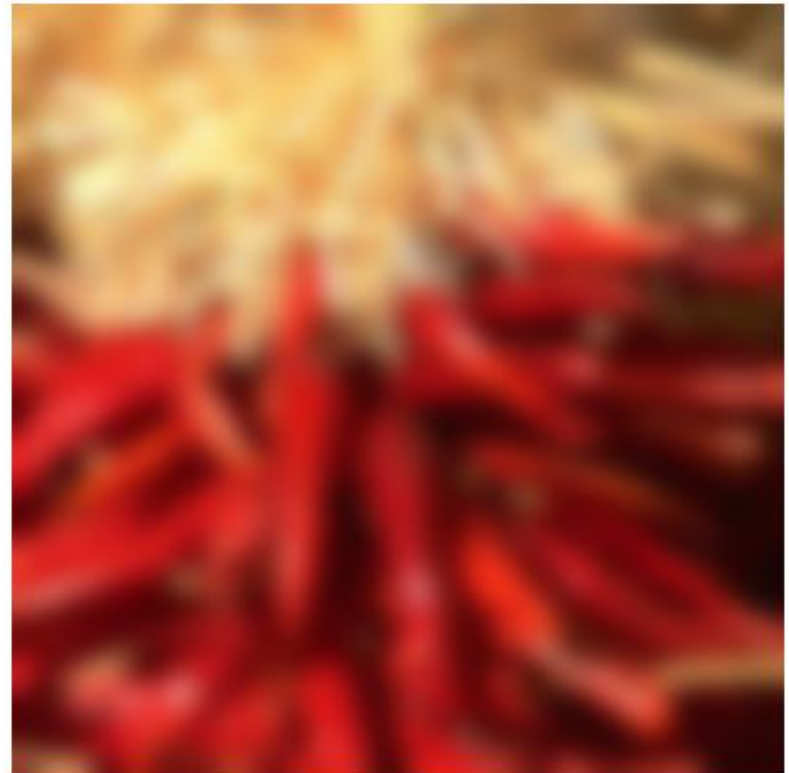
What is the size of the output?



Boundary issues

What about near the edge?

- the filter window falls off the edge of the image
- need to extrapolate
- methods:
 - clip filter (black)
 - wrap around
 - copy edge
 - reflect across edge



Boundary issues

What about near the edge?

- the filter window falls off the edge of the image
- need to extrapolate
- [cv::BorderTypes](#)

BORDER_CONSTANT	clip filter (black)
BORDER_REPLICATE	wrap around
BORDER_REFLECT	reflect across edge
BORDER_WRAP	wrap around

Gaussian filter

- What if we want nearest neighboring pixels to have the most influence on the output?

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$F[x, y]$

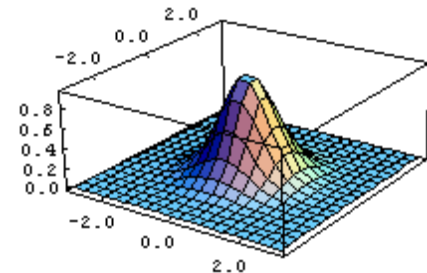
$\frac{1}{16}$

1	2	1
2	4	2
1	2	1

$H[u, v]$

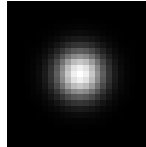
This kernel is an approximation of a 2d Gaussian function:

$$h(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{\sigma^2}}$$



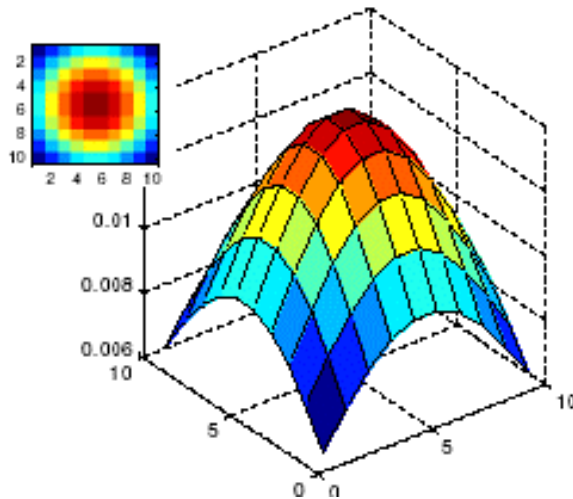
- Removes high-frequency components from the image (“low-pass filter”).

Smoothing with a Gaussian

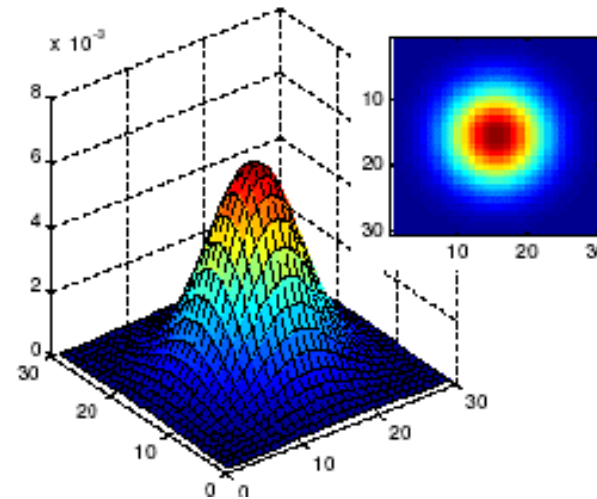


Gaussian filters

- What parameters matter here?
- **Size** of kernel or mask
 - Note, Gaussian function has infinite support, but discrete filters use finite kernels



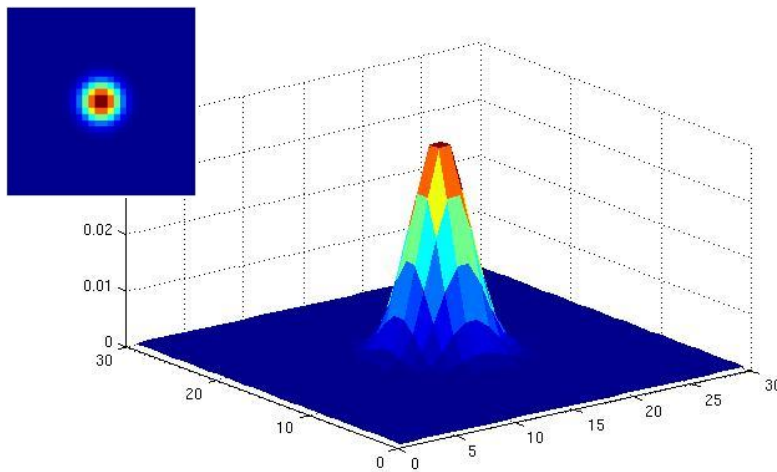
$\sigma = 5$ with
10 x 10
kernel



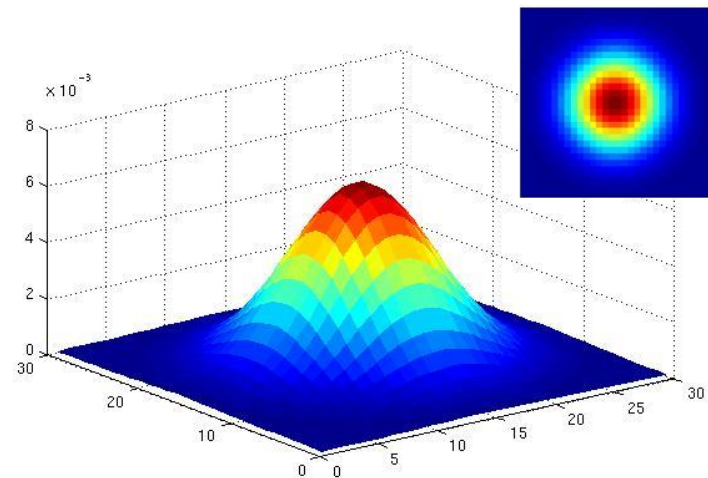
$\sigma = 5$ with
30 x 30
kernel

Gaussian filters

- What parameters matter here?
- **Variance** of Gaussian: determines extent of smoothing



$\sigma = 2$ with
 30×30
kernel



$\sigma = 5$ with
 30×30
kernel

OpenCV

2. Gaussian Filtering

In this approach, instead of a box filter consisting of equal filter coefficients, a Gaussian kernel is used. It is done with the function, **cv2.GaussianBlur()**. We should specify the width and height of the kernel which should be positive and odd. We also should specify the standard deviation in the X and Y directions, **sigmaX** and **sigmaY** respectively. If only **sigmaX** is specified, **sigmaY** is taken as equal to **sigmaX**. If both are given as zeros, they are calculated from the kernel size. Gaussian filtering is highly effective in removing Gaussian noise from the image.

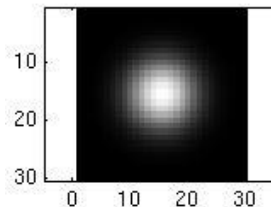
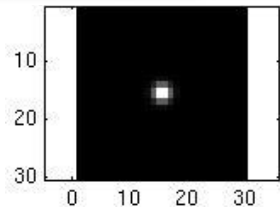
If you want, you can create a Gaussian kernel with the function, **cv2.getGaussianKernel()**.

The above code can be modified for Gaussian blurring:

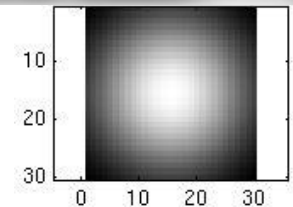
```
>> noiseReducedGaussianBlur =  
cv2.GaussianBlur(imgGreenNoise, (5,5), 0)
```

Smoothing with a Gaussian

Parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel, and controls the amount of smoothing.



...

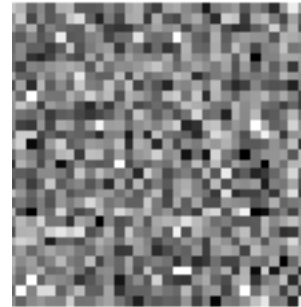


For Sigma 1, 4, and 7

Keeping the two Gaussians in play straight...

Wider smoothing kernel →

$\sigma=0.2$

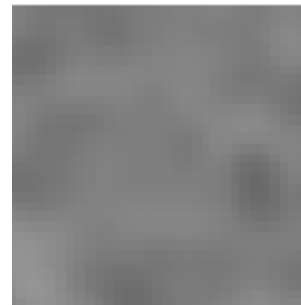


no
smoothing

$\sigma=1$ pixel



$\sigma=2$ pixels



Properties of smoothing filters

- Smoothing
 - Values positive
 - Sum to 1 $\rightarrow$ constant regions same as input
 - Amount of smoothing proportional to mask size
 - Remove “high-frequency” components; “low-pass” filter

Predict the outputs using correlation filtering


$$\begin{matrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{matrix} = ?$$


$$\begin{matrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{matrix} = ?$$


$$\begin{matrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{matrix} - \frac{1}{9} \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix} = ?$$

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0

?

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0



**Filtered
(no change)**

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0

?

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0



**Shifted left
by 1 pixel
with
correlation**

Practice with linear filters



Original

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

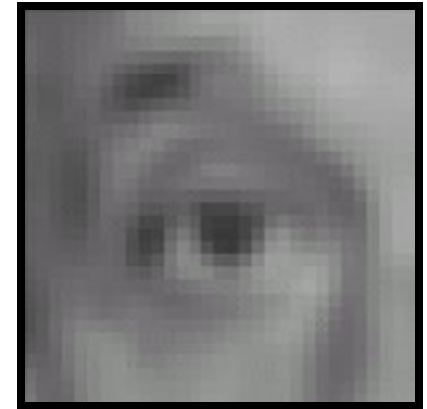
Practice with linear filters



Original

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1



**Blur (with a
box filter)**

Practice with linear filters



Original

0	0	0
0	2	0
0	0	0

—

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

Practice with linear filters



Original

0	0	0
0	2	0
0	0	0

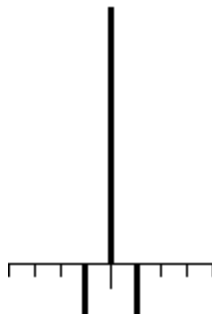
−

$\frac{1}{9}$

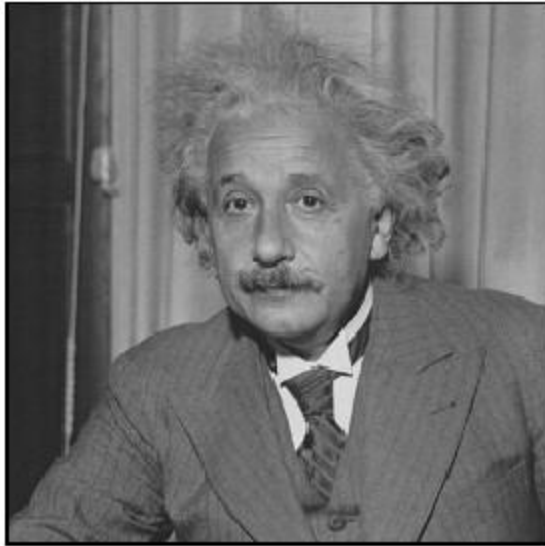
1	1	1
1	1	1
1	1	1



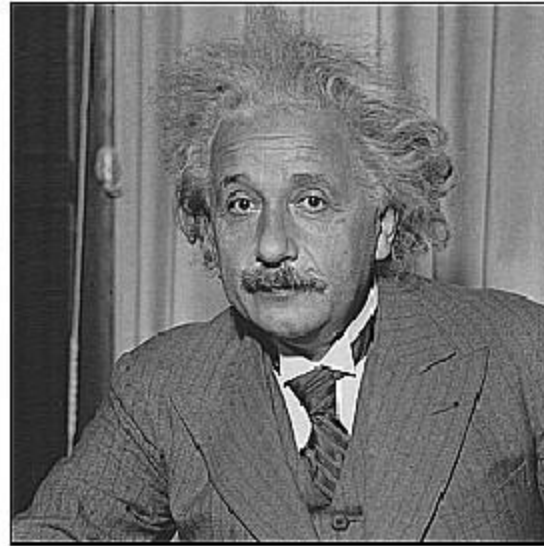
Sharpening filter:
accentuates differences
with local average



Filtering examples: sharpening



before



after

Convolution

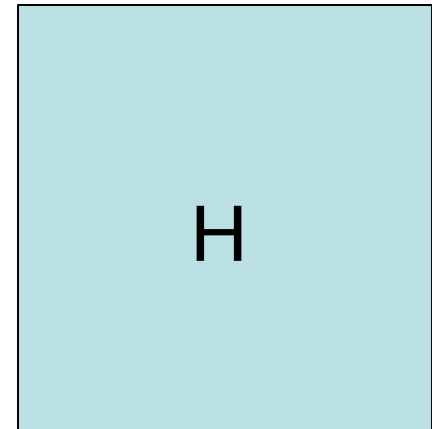
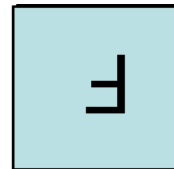
- Convolution:
 - Flip the filter in both dimensions (bottom to top, right to left)
 - Then apply cross-correlation

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

$$G = H \star F$$



*Notation for
convolution
operator*



Properties of convolution (and also correlation)

- **Shift invariant:**
 - Operator behaves the same everywhere, i.e. the value of the output depends on the pattern in the image neighborhood, not the position of the neighborhood.
- **Superposition:**
 - $h * (f1 + f2) = (h * f1) + (h * f2)$

Properties of convolution

- Commutative:

$$f * g = g * f$$

- Associative

$$(f * g) * h = f * (g * h)$$

- Distributes over addition

$$f * (g + h) = (f * g) + (f * h)$$

- Scalars factor out

$$kf * g = f * kg = k(f * g)$$

- Identity:

$$\text{unit impulse } e = [\dots, 0, 0, 1, 0, 0, \dots]. \quad f * e = f$$

Separability

- In some cases, filter is separable, and we can factor into two steps:
 - Convolve all rows with a 1D filter
 - Convolve all columns with a 1D filter

$$\begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{3} & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & \frac{1}{3} & 0 \end{bmatrix} \circ \begin{bmatrix} 0 & 0 & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & 0 \end{bmatrix}$$

Separability of the Gaussian filter

$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

Separability example

**2D convolution
(center location only)**

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 2 & 3 & 3 \\ 3 & 5 & 5 \\ 4 & 4 & 6 \end{bmatrix}$$

**The filter factors
into a product of 1D
filters:**

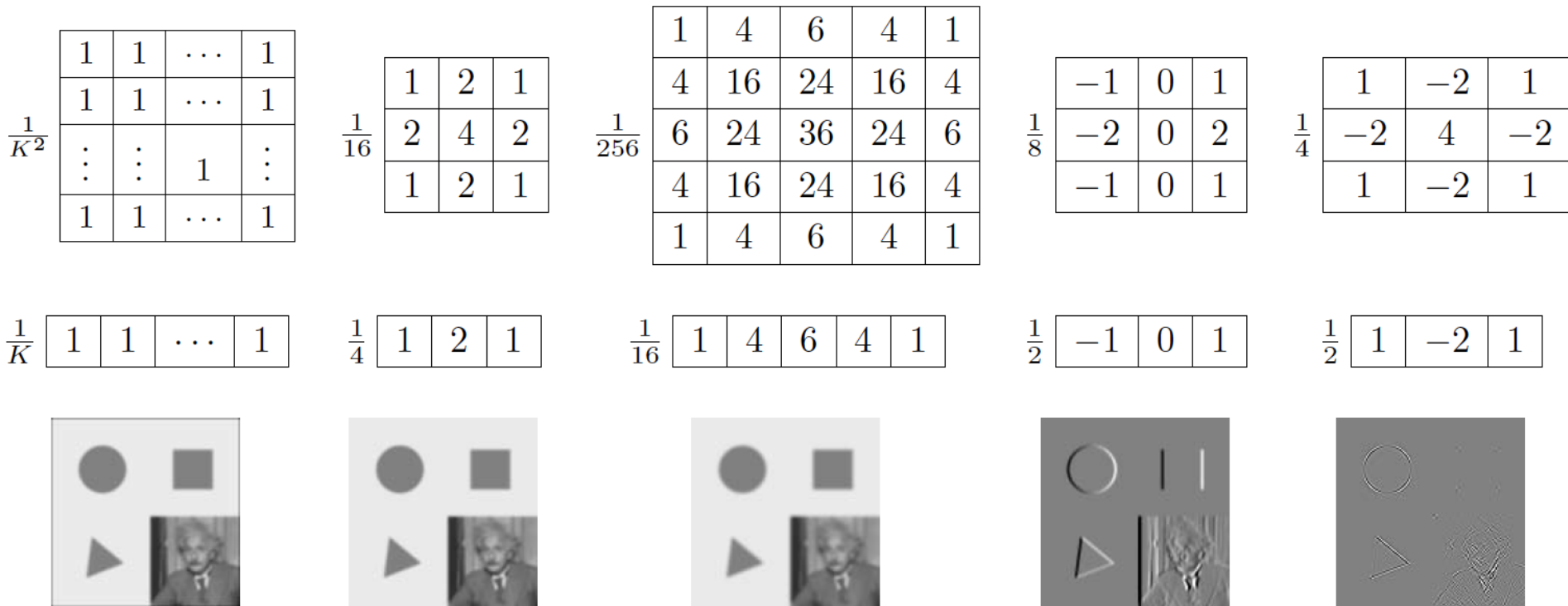
$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

**Perform convolution
along rows:**

$$\begin{bmatrix} 1 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 2 & 3 & 3 \\ 3 & 5 & 5 \\ 4 & 4 & 6 \end{bmatrix} = \begin{bmatrix} & 11 & \\ & 18 & \\ & 18 & \end{bmatrix}$$

**Followed by convolution
along the remaining column:**

Textbook Example (Figure 3.14)



(a) box, $K = 5$

(b) bilinear

(c) “Gaussian”

(d) Sobel

(e) corner

Figure 3.14 Separable linear filters: For each image (a)–(e), we show the 2D filter kernel (top), the corresponding horizontal 1D kernel (middle), and the filtered image (bottom). The filtered Sobel and corner images are signed, scaled up by $2\times$ and $4\times$, respectively, and added to a gray offset before display.

Separability

- Why is separability useful in practice?
- The process of performing a convolution requires K^2 (multiply-add) operations per pixel where K is the size (width or height) of the convolution kernel
- This operation can be significantly sped up by first performing a one-dimensional horizontal convolution followed by a one-dimensional vertical convolution
 - requires a total of $2K$ operations per pixel).

Some practical matters

How big should the filter be?

- Values at edges should be near zero
- Rule of thumb for Gaussian: set filter half-width to about 3σ

Effect of smoothing filters

5x5

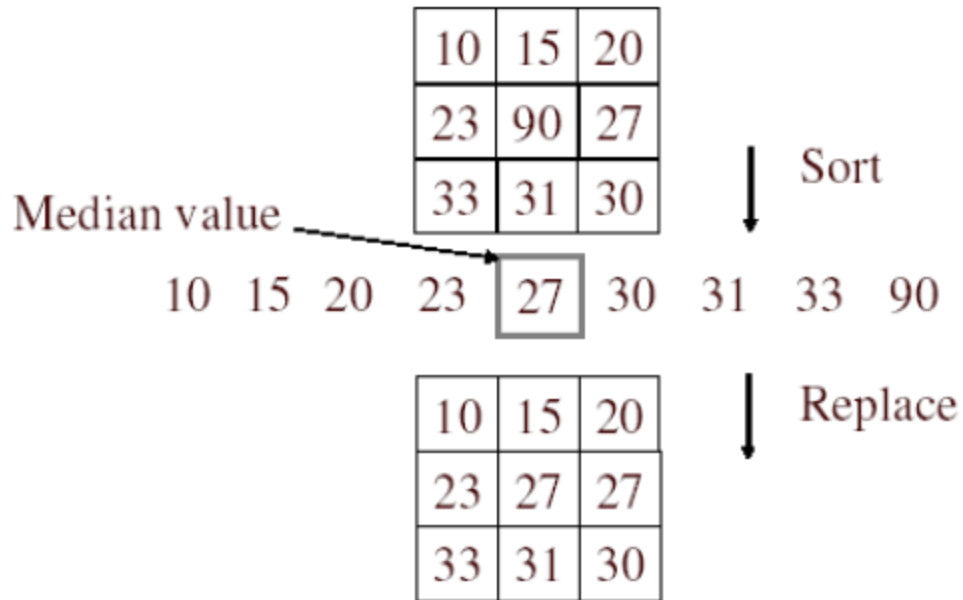


Additive Gaussian noise



Salt and pepper noise

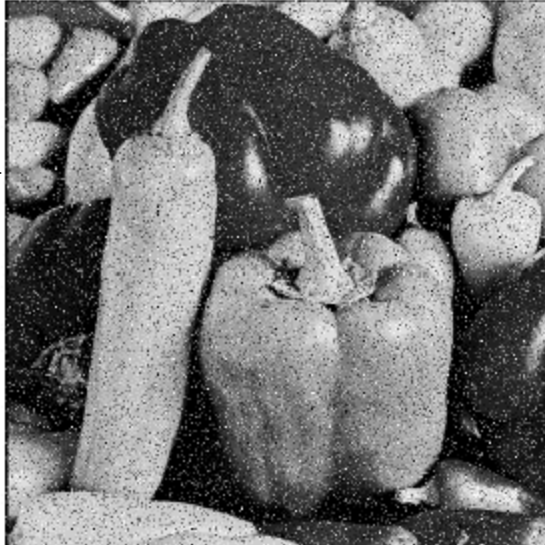
Median filter



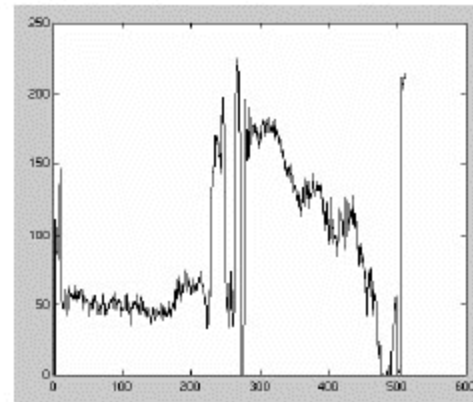
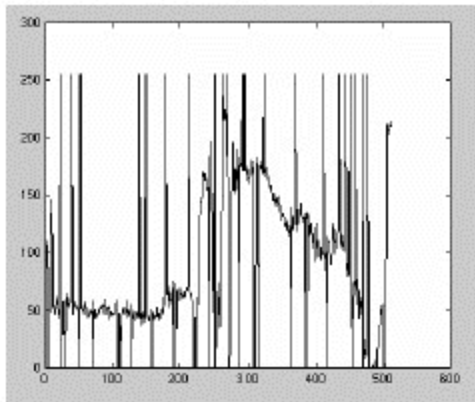
- No new pixel values introduced
- Removes spikes: good for impulse, salt & pepper noise
- Non-linear filter

Median filter

Salt and
pepper
noise



Median
filtered

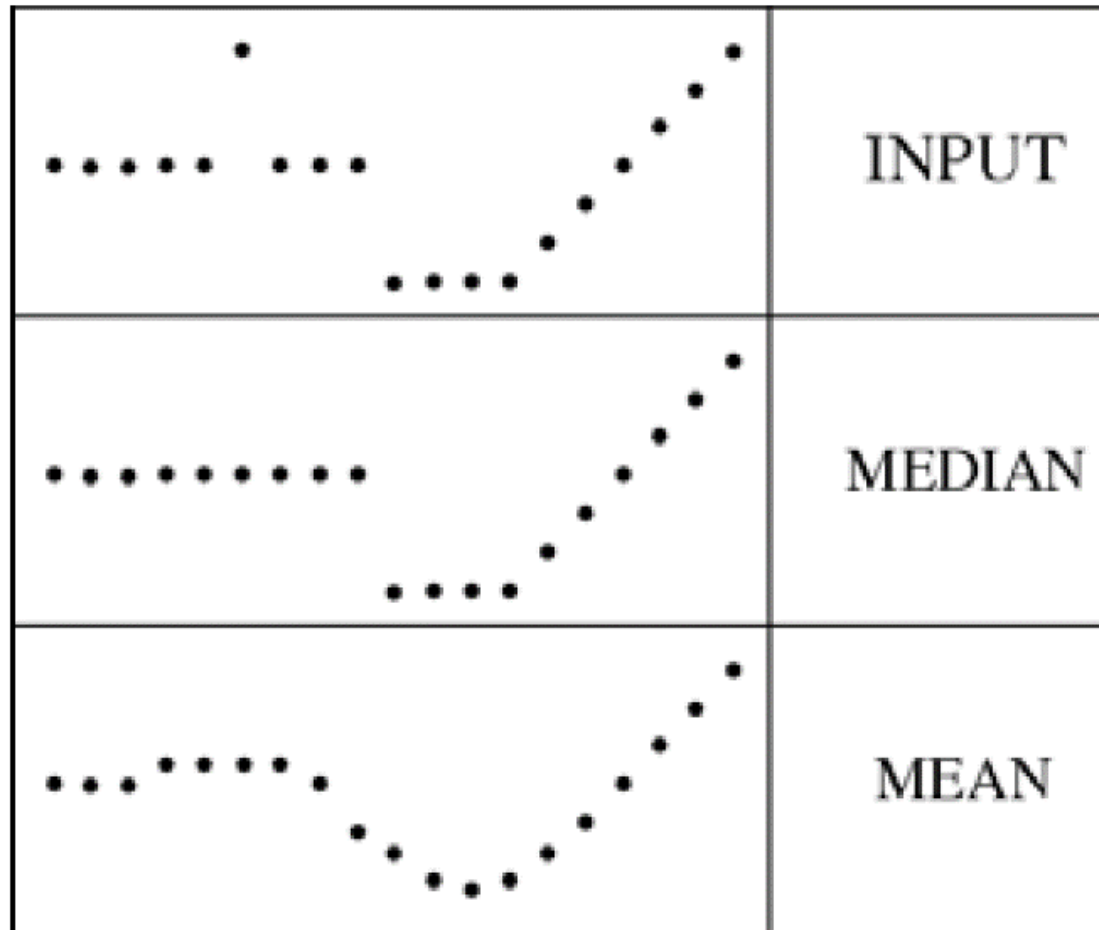


Plots of a row of the image

OpenCV: `median = cv2.medianBlur(img,5)`

Median filter

- Median filter is edge preserving



Summary

- Image “noise”
- Linear filters and convolution useful for
 - Enhancing images (smoothing, removing noise)
 - Box filter
 - Gaussian filter
 - Impact of scale / width of smoothing filter
 - Detecting features (next time)
- Separable filters more efficient
- Median filter: a non-linear filter, edge-preserving